```
# ========================================
# 1. Import Libraries
# ========================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# ========================================
# 2. Load Dataset as DataFrame
# ========================================
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# ========================================
# 3. Feature-Target Split
# ========================================
X = df.drop("Price", axis=1)
y = df["Price"]

# ========================================
# 4. Train-Test Split
# ========================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# ========================================
# 5. Feature Scaling (VERY IMPORTANT)
# ========================================
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ========================================
# 6. Build Multilayer Model
# ========================================
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)  # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

model.summary()

# ========================================
# 7. Train Model
# ========================================
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=32,
    verbose=1
)

# ========================================
# 8. Evaluate Model
# ========================================
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# ========================================
# 9. Plot Training Curve
# ========================================
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
```

```
plt.ylabel("MSE Loss")
plt.show()

# ========================================
# 10. Actual vs Predicted Plot
# ========================================
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()
```

```
Dataset Shape: (20640, 9)
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85

   Longitude  Price
0    -122.23  4.526
1    -122.22  3.585
2    -122.24  3.521
3    -122.25  3.413
4    -122.25  3.422
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` arg
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	1,152
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 32)	2,080
dense_3 (Dense)	(None, 1)	33

```
 Total params: 11,521 (45.00 KB)
 Trainable params: 11,521 (45.00 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 0.6400 - mae: 0.5482 - val_loss: 0.4129 - val_mae: 0.4476
Epoch 2/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.3934 - mae: 0.4341 - val_loss: 0.3870 - val_mae: 0.4423
Epoch 3/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 0.3458 - mae: 0.4141 - val_loss: 0.3667 - val_mae: 0.4268
Epoch 4/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3292 - mae: 0.4007 - val_loss: 0.3714 - val_mae: 0.4515
Epoch 5/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3255 - mae: 0.3946 - val_loss: 0.3329 - val_mae: 0.4001
Epoch 6/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3225 - mae: 0.3859 - val_loss: 0.3575 - val_mae: 0.3955
Epoch 7/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2995 - mae: 0.3777 - val_loss: 0.3346 - val_mae: 0.4145
Epoch 8/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2930 - mae: 0.3744 - val_loss: 0.3326 - val_mae: 0.3880
Epoch 9/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2898 - mae: 0.3716 - val_loss: 0.3169 - val_mae: 0.3782
Epoch 10/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2842 - mae: 0.3669 - val_loss: 0.3313 - val_mae: 0.3840
Epoch 11/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2815 - mae: 0.3651 - val_loss: 0.3221 - val_mae: 0.3814
Epoch 12/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2785 - mae: 0.3623 - val_loss: 0.3245 - val_mae: 0.3833
Epoch 13/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2803 - mae: 0.3609 - val_loss: 0.3062 - val_mae: 0.3730
Epoch 14/100
```

```
print(history.history.keys())
```

```
Epoch 17/100
dict_keys(['loss', 'mae', 'val_loss', 'val_mae'])
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2628 - mae: 0.3510 - val_loss: 0.2981 - val_mae: 0.3667
```

```
# for batch 64
history_64 = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=64,
    verbose=0
)
```

```
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2496 - mae: 0.3406 - val_loss: 0.3106 - val_mae: 0.4038
```

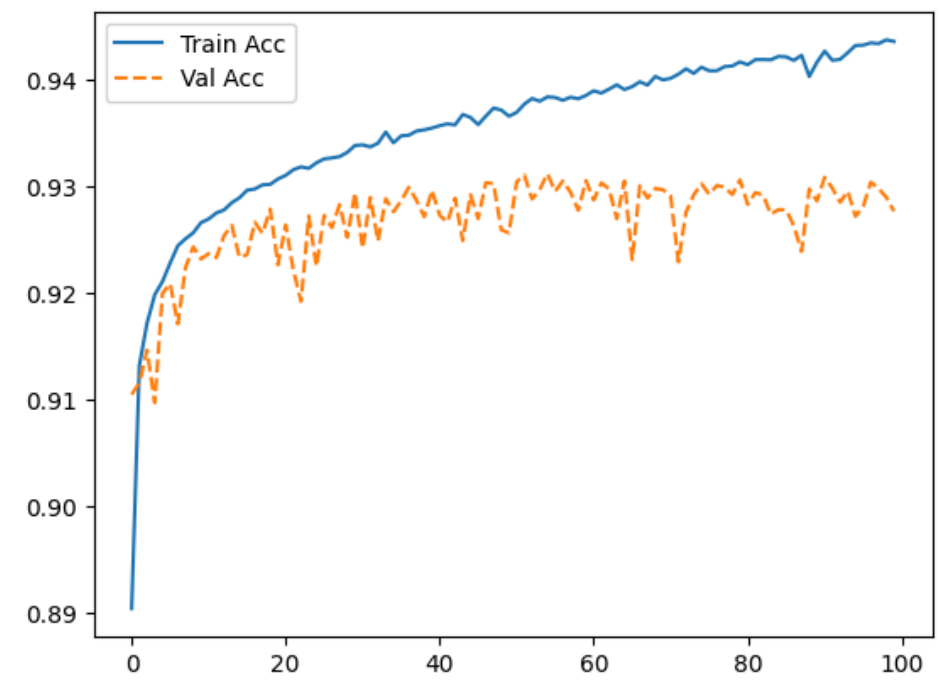
```
# for batch 128
history_128 = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=128,
    verbose=0
)
```

```
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2396 - mae: 0.3338 - val_loss: 0.2902 - val_mae: 0.3738
```

```
print(history.history.keys())
```

```
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2360 - mae: 0.3303 - val_loss: 0.3030 - val_mae: 0.3790
dict_keys(['loss', 'mae', 'val_loss', 'val_mae'])
Epoch 32/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2369 - mae: 0.3312 - val_loss: 0.2767 - val_mae: 0.3551
```

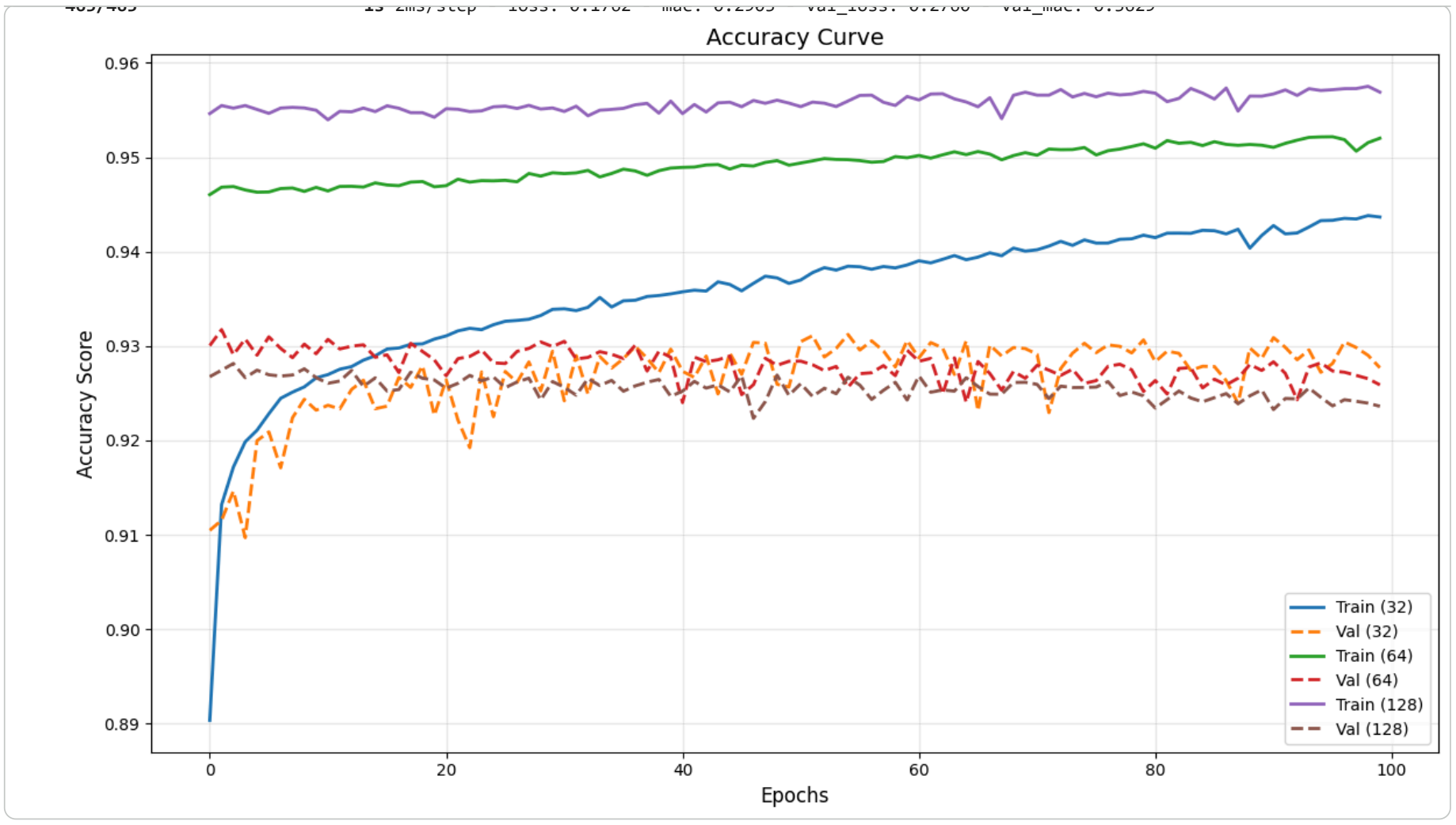
```
max_val = np.max(y_train)

train_acc_like = 1 - (np.array(history.history['mae']) / max_val)
val_acc_like = 1 - (np.array(history.history['val_mae']) / max_val)

plt.plot(train_acc_like, '-', label='Train Acc')
plt.plot(val_acc_like, '--', label='Val Acc')
plt.legend()
plt.show()
```




```
val_loss: 0.2744 - val_mae: 0.3564
val_loss: 0.2744 - val_mae: 0.3641
val_loss: 0.2771 - val_mae: 0.3517
val_loss: 0.2817 - val_mae: 0.3634
val_loss: 0.2946 - val_mae: 0.3666
val_loss: 0.2768 - val_mae: 0.3553
val_loss: 0.2860 - val_mae: 0.3754
val_loss: 0.2777 - val_mae: 0.3538
val_loss: 0.2901 - val_mae: 0.3650
val_loss: 0.2703 - val_mae: 0.3481
val_loss: 0.2751 - val_mae: 0.3485
val_loss: 0.2940 - val_mae: 0.3702
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2143 - mae: 0.3168 - val_loss: 0.2905 - val_mae: 0.3717
```

```
import numpy as np
import matplotlib.pyplot as plt

max_val = np.max(y_train)

# Convert MAE to accuracy-like
train_32 = 1 - (np.array(history.history['mae']) / max_val)
val_32   = 1 - (np.array(history.history['val_mae']) / max_val)

train_64 = 1 - (np.array(history_64.history['mae']) / max_val)
val_64   = 1 - (np.array(history_64.history['val_mae']) / max_val)

train_128 = 1 - (np.array(history_128.history['mae']) / max_val)
val_128   = 1 - (np.array(history_128.history['val_mae']) / max_val)


plt.figure(figsize=(12,7))

# Batch 32
plt.plot(train_32, '-',  linewidth=2, label='Train (32)')
plt.plot(val_32,   '--', linewidth=2, label='Val (32)')

# Batch 64
plt.plot(train_64, '-',  linewidth=2, label='Train (64)')
plt.plot(val_64,   '--', linewidth=2, label='Val (64)')

# Batch 128
plt.plot(train_128, '-',  linewidth=2, label='Train (128)')
plt.plot(val_128,   '--', linewidth=2, label='Val (128)')


plt.title("Accuracy Curve", fontsize=14)
plt.xlabel("Epochs", fontsize=12)
plt.ylabel("Accuracy Score", fontsize=12)

plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1826 - mae: 0.2946 - val_loss: 0.2851 - val_mae: 0.3619
Epoch 74/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1841 - mae: 0.2967 - val_loss: 0.2794 - val_mae: 0.3538
Epoch 75/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1799 - mae: 0.2938 - val_loss: 0.2677 - val_mae: 0.3486
Epoch 76/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1825 - mae: 0.2955 - val_loss: 0.2773 - val_mae: 0.3535
Epoch 77/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1813 - mae: 0.2955 - val_loss: 0.2725 - val_mae: 0.3494
Epoch 78/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1823 - mae: 0.2935 - val_loss: 0.2682 - val_mae: 0.3502
Epoch 79/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1810 - mae: 0.2932 - val_loss: 0.2692 - val_mae: 0.3537
Epoch 80/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1768 - mae: 0.2913 - val_loss: 0.2646 - val_mae: 0.3468
Epoch 81/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1787 - mae: 0.2926 - val_loss: 0.2752 - val_mae: 0.3583
Epoch 82/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1763 - mae: 0.2902 - val_loss: 0.2725 - val_mae: 0.3527
Epoch 83/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1750 - mae: 0.2902 - val_loss: 0.2876 - val_mae: 0.3539
Epoch 84/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1762 - mae: 0.2903 - val_loss: 0.2760 - val_mae: 0.3629
```




```
Test MAE: 0.3562500774860382
```